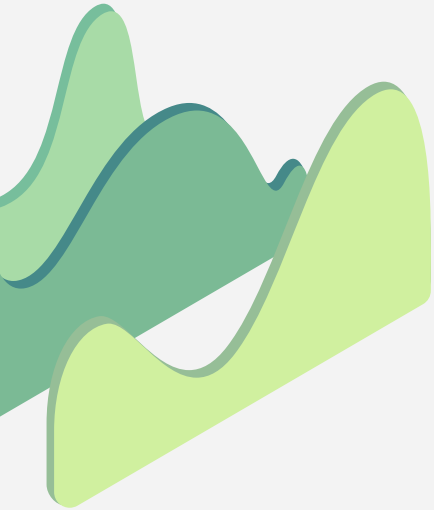
200Lab Education

Python for Data Analysis

Nga Vu - Data Mentor





tailieusharefree.blogspot.com



200Lab Education

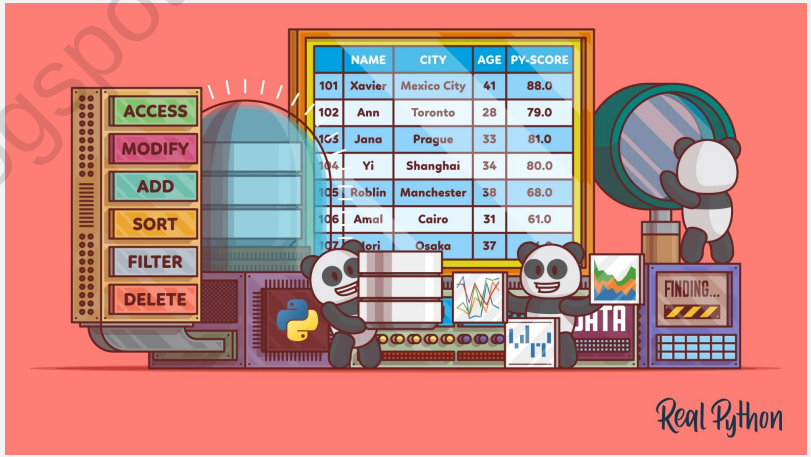
Pandas

Yet another common library

tailieusharefree.blogspot.com

What is Pandas ?

- **Pandas** is an open-source, BSD-licensed library written in Python Language.
- **Pandas** makes data cleaning and analysis fast and easy in Python.
- The biggest difference is that **pandas** is designed for working with tabular or heterogeneous data. **NumPy**, by contrast, is best suited for working with homogeneous numerical array data
- Whenever you see **pd.** in code, it's referring to pandas.




```
pip install pandas
```

```
import pandas as pd
```

tailieusharefree.blogspot.com

Pandas vs Numpy

Pandas	Numpy
When we have to work on Tabular data, we prefer the pandas module	When we have to work on Numerical data, we prefer the numpy module
The powerful tools of pandas are Data frame and Series	Whereas the powerful tool of numpy is Arrays
Pandas consume more memory	Numpy is memory efficient
Indexing of the pandas series is very slow as compared to numpy arrays	Indexing of numpy Arrays is very fast
Pandas offers 2d table object called DataFrame	Numpy is capable of providing multi-dimensional arrays

tailieusharefree.blogspot.com

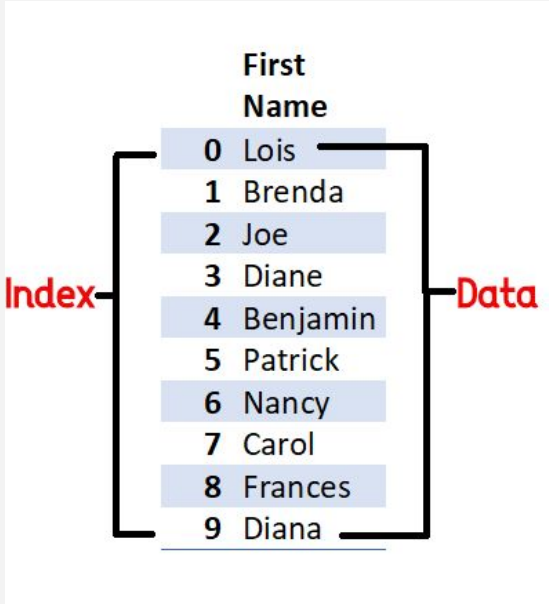
Series

Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.). The axis labels are collectively referred to as the index

```
s = pd.Series(data, index=index)
```

data can be many different things:

- a Python dict
- an ndarray
- a Python list
- a scalar value (like 5)



tailieusharefree.blogspot.com

Create a Series

If data is an **ndarray**, index must be the same length as data. If no index is passed, one will be created having values [0, ..., len(data) - 1]

```
In [3]: s = pd.Series(np.random.randn(5), index=["a", "b", "c", "d", "e"])

In [4]: s
Out[4]:
a    0.469112
b   -0.282863
c   -1.509059
d   -1.135632
e    1.212112
dtype: float64
```

```
In [5]: s.index
Out[5]: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')

In [6]: pd.Series(np.random.randn(5))
Out[6]:
0   -0.173215
1    0.119209
2   -1.044236
3   -0.861849
4   -2.104569
dtype: float64
```

Series can be instantiated from **dicts**

```
In [7]: d = {"b": 1, "a": 0, "c": 2}

In [8]: pd.Series(d)
Out[8]:
b    1
a    0
c    2
dtype: int64
```

```
In [9]: d = {"a": 0.0, "b": 1.0, "c": 2.0}

In [10]: pd.Series(d)
Out[10]:
a    0.0
b    1.0
c    2.0
dtype: float64

In [11]: pd.Series(d, index=["b", "c", "d", "a"])
Out[11]:
b    1.0
c    2.0
d    NaN
a    0.0
dtype: float64
```

tailieusharefree.blogspot.com

Create a Series

Create Series from Python list

```
a = pd.Series([1,2,3])
a
```

```
0    1
1    2
2    3
dtype: int64
```

If data is a **scalar** value, an index must be provided. The value will be repeated to match the length of index

```
In [12]: pd.Series(5.0, index=["a", "b", "c", "d", "e"])
Out[12]:
a    5.0
b    5.0
c    5.0
d    5.0
e    5.0
dtype: float64
```

tailieusharetree.blogspot.com

Create a Series

String index vs Integer index

```
a = pd.Series([1,2,3],index=[1,2,3])
a[1]
```

```
1
```

```
a = pd.Series([1,2,3],index=[1,2,3])
a[0]
```

```
---------------------------------------
KeyError
```

```
a = pd.Series([1,2,3],index=['1','2','3'])
a['1']
```

```
1
```

```
a = pd.Series([1,2,3],index=['1','2','3'])
a[0]
```

```
1
```

Series acts very similarly to a ndarray, and is a valid argument to most NumPy functions. However, operations such as slicing will also slice the index

```
In [13]: s[0]
Out[13]: 0.4691122999071863

In [14]: s[:3]
Out[14]:
a    0.469112
b   -0.282863
c   -1.509059
dtype: float64
```

```
In [17]: np.exp(s)
Out[17]:
a    1.598575
b    0.753623
c    0.221118
d    0.321219
e    3.360575
dtype: float64
```

```
In [15]: s[s > s.median()]
Out[15]:
a    0.469112
e    1.212112
dtype: float64

In [16]: s[[4, 3, 1]]
Out[16]:
e    1.212112
d   -1.135632
b   -0.282863
dtype: float64
```

tailieusharefree.blogspot.com

Create a Series

Series can also have a **name** attribute

```
In [32]: s = pd.Series(np.random.randn(5), name="something")

In [33]: s
Out[33]:
0   -0.494929
1    1.071804
2    0.721555
3   -0.706771
4   -1.039575
Name: something, dtype: float64

In [34]: s.name
Out[34]: 'something'
```

You can rename a Series with the **pandas.Series.rename()** method

```
In [35]: s2 = s.rename("different")

In [36]: s2.name
Out[36]: 'different'
```

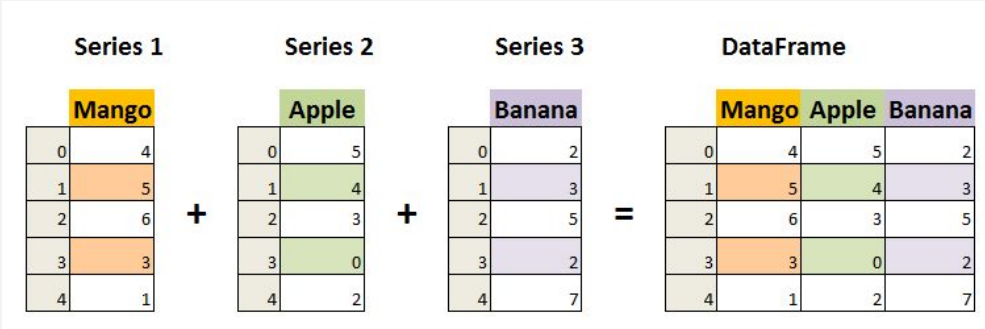
tailieusharefree.blogspot.com

DataFrame

DataFrame is a 2-dimensional labeled data structure with columns of potentially different types: like a spreadsheet or SQL table, or **a dict of Series objects**. It is generally the most commonly used pandas object.

Like Series, DataFrame accepts many different kinds of input:

- Numpy Array
- Python List
- Python Dict
- Series
- Another DataFrame



tailieusharefree.blogspot.com

Create a DataFrame

From dict of Series or dicts

The resulting index will be the union of the indexes of the various Series.

If there are any nested dicts, these will first be converted to Series. If no columns are passed, the columns will be the ordered list of dict keys

```
In [37]: d = {
   ....:     "one": pd.Series([1.0, 2.0, 3.0], index=["a", "b", "c"]),
   ....:     "two": pd.Series([1.0, 2.0, 3.0, 4.0], index=["a", "b", "c", "d"]),
   ....: }
   ....:

In [38]: df = pd.DataFrame(d)

In [39]: df
Out[39]:
   one  two
a  1.0  1.0
b  2.0  2.0
c  3.0  3.0
d  NaN  4.0
In [40]: pd.DataFrame(d, index=["d", "b", "a"])
Out[40]:
   one  two
d  NaN  4.0
b  2.0  2.0
a  1.0  1.0

In [41]: pd.DataFrame(d, index=["d", "b", "a"], columns=["two", "three"])
Out[41]:
   two three
d  4.0   NaN
b  2.0   NaN
a  1.0   NaN
```

```
In [42]: df.index
Out[42]: Index(['a', 'b', 'c', 'd'], dtype='object')

In [43]: df.columns
Out[43]: Index(['one', 'two'], dtype='object')
```

tailieusharefree.blogspot.com

Create a DataFrame

From dict of ndarrays / lists

The ndarrays must all be the same length. If an index is passed, it must clearly also be the same length as the arrays.

If no index is passed, the result will be range(n), where n is the array length

```
In [44]: d = {"one": [1.0, 2.0, 3.0, 4.0], "two": [4.0, 3.0, 2.0, 1.0]}

In [45]: pd.DataFrame(d)
Out[45]:
   one  two
0  1.0  4.0
1  2.0  3.0
2  3.0  2.0
3  4.0  1.0

In [46]: pd.DataFrame(d, index=["a", "b", "c", "d"])
Out[46]:
   one  two
a  1.0  4.0
b  2.0  3.0
c  3.0  2.0
d  4.0  1.0
```

tailieusharefree.blogspot.com

200Lab Education

Create a DataFrame



From a list of dicts

```
In [52]: data2 = [{"a": 1, "b": 2}, {"a": 5, "b": 10, "c": 20}]

In [53]: pd.DataFrame(data2)
Out[53]:
   a   b     c
0  1   2   NaN
1  5  10  20.0

In [54]: pd.DataFrame(data2, index=["first", "second"])
Out[54]:
        a   b     c
first   1   2   NaN
second  5  10  20.0

In [55]: pd.DataFrame(data2, columns=["a", "b"])
Out[55]:
   a   b
0  1   2
1  5  10
```

From a list of tuples

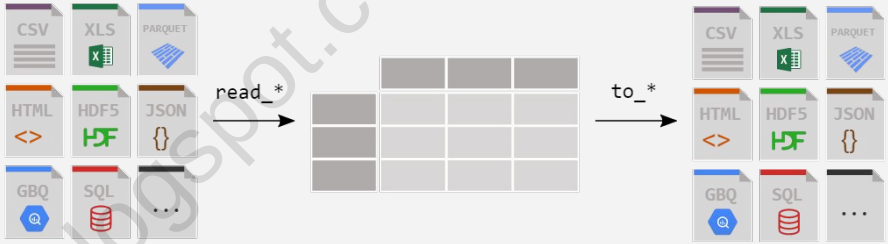
```
data = [('Peter', 18, 7),
        ('Riff', 15, 6),
        ('John', 17, 8),
        ('Michel', 18, 7),
        ('Sheli', 17, 5) ]
df = pd.DataFrame(data, columns =['Name', 'Age', 'Score'])
df
```

	Name	Age	Score
0	Peter	18	7
1	Riff	15	6
2	John	17	8
3	Michel	18	7
4	Sheli	17	5

tailieusharefree.blogspot.com

How do I read and write tabular data ?

Pandas provides the read_csv() function to read data stored as a csv file into a pandas DataFrame.



Pandas supports many different file formats or data sources out of the box (csv, excel, sql, json, parquet, …), each of them with the prefix **read_***

```
import pandas as pd
```

```
titanic = pd.read_csv("data/titanic.csv")
```

```
In [3]: titanic
Out[3]:
     PassengerId  Survived  Pclass                                               Name
0              1         0       3                            Braund, Mr. Owen Harris
1              2         1       1  Cumings, Mrs. John Bradley (Florence Briggs Th...
2              3         1       3                             Heikkinen, Miss. Laina
3              4         1       1       Futrelle, Mrs. Jacques Heath (Lily May Peel)
4              5         0       3                           Allen, Mr. William Henry
```

taileusharefree.blogspot.com

How do I read and write tabular data ?

Whereas **read_*** functions are used to read data to pandas, the **to_*** methods are used to store data.

The to_excel() method stores the data as an excel file.

```
titanic.to_excel("titanic.xlsx", sheet_name="passengers", index=False)
```

tailieusharefree.blogspot.com

Filter a DataFrame

Single condition Filtering

Extracting a **subset** of data is perhaps the most common operation in data analysis. A **subset** is a portion of a larger data set that fits some kind of condition

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical

```
df['EducationField']=='Medical'
```

```
0       False
1       False
2       False
3       False
4        True
        ...
1465     True
1466     True
1467    False
1468     True
1469     True
Name: EducationField, Length: 1470, dtype: bool
```

```
df[df['EducationField']=='Medical']
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical
6	59	No	Travel_Rarely	1324	Research & Development	3	3	Medical
9	36	No	Travel_Rarely	1299	Research & Development	27	3	Medical
10	35	No	Travel_Rarely	809	Research & Development	16	3	Medical
13	34	No	Travel_Rarely	1346	Research & Development	19	2	Medical

tailieusharefree.blogspot.com

Filter a DataFrame

Single condition Filtering

```
df[df['Gender']=='Female']
```

	Age	Attrition	BusinessTravel	DailyRate	Department
0	41	Yes	Travel_Rarely	1102	Sales
3	33	No	Travel_Frequently	1392	Research & Development
6	59	No	Travel_Rarely	1324	Research & Development
11	29	No	Travel_Rarely	153	Research & Development

```
df[df['Age']>40]
```

	Age	Attrition	BusinessTravel	DailyRate	Department
0	41	Yes	Travel_Rarely	1102	Sales
1	49	No	Travel_Frequently	279	Research & Development
6	59	No	Travel_Rarely	1324	Research & Development
18	53	No	Travel_Rarely	1219	Sales
25	53	No	Travel_Rarely	1282	Research & Development

tailieushaefree.blogspot.com

Filter a DataFrame

Multiple conditions Filtering

1. AND condition

Condition 1	Condition 2	Evaluation
True	True	True
True	False	False
False	True	False
False	False	False

```
df[(df['Gender']=='Female') & (df['EducationField']=='Medical')]
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education
6	59	No	Travel_Rarely	1324	Research & Development	3	3
28	44	No	Travel_Rarely	477	Research & Development	7	4
35	43	No	Travel_Rarely	1273	Research & Development	2	2
44	30	No	Travel_Frequently	721	Research & Development	1	2
57	35	No	Travel_Rarely	1142	Research & Development	23	4

```
df[(df['Gender']=='Female') & (df['EducationField']=='Medical') & (df['Age']>40)]
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Edu
6	59	No	Travel_Rarely	1324	Research & Development	3	
28	44	No	Travel_Rarely	477	Research & Development	7	
35	43	No	Travel_Rarely	1273	Research & Development	2	
62	50	No	Travel_Rarely	989	Research & Development	7	

tailieusharefree.blogspot.com

Filter a DataFrame

Multiple conditions Filtering

2. OR condition

Condition 1	Condition 2	Evaluation
True	True	True
True	False	True
False	True	True
False	False	False

4. Inversion with ~ (NOT)

```
df[(df['Gender']=='Female') | (df['EducationField']=='Medical')]
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome
0	41	Yes	Travel_Rarely	1102	Sales	1
3	33	No	Travel_Frequently	1392	Research & Development	3
4	27	No	Travel_Rarely	591	Research & Development	2
6	59	No	Travel_Rarely	1324	Research & Development	3

```
df[~(df['Age']>40)]
```

	Age	Attrition	BusinessTravel	DailyRate	Department
2	37	Yes	Travel_Rarely	1373	Research & Development
3	33	No	Travel_Frequently	1392	Research & Development
4	27	No	Travel_Rarely	591	Research & Development
5	32	No	Travel_Frequently	1005	Research & Development
7	30	No	Travel_Rarely	1358	Research & Development

tailieusharefree.blogspot.com

Filter a DataFrame

Multiple conditions Filtering

Equivalent methods

Operation	Arithmetic syntax	Method syntax
Equality	`employees["Team"] == "Marketing"`	`employees["Team"].eq("Marketing")`
Inequality	`employees["Team"] != "Marketing"`	`employees["Team"].ne("Marketing")`
Less than	`employees["Salary"] < 100000`	`employees["Salary"].lt(100000)`
Less than or equal to	`employees["Salary"] <= 100000`	`employees["Salary"].le(100000)`
Greater than	`employees["Salary"] > 100000`	`employees["Salary"].gt(100000)`
Greater than or equal to	`employees["Salary"] >= 100000`	`employees["Salary"].ge(100000)`

tailieusharefree.blogspot.com

Filter a DataFrame

isin() Method

```
df[(df['EducationField']=='Medical') | (df['EducationField']=='Life Sciences  ')]
```

```
df[df['EducationField'].isin(['Medical','Life Sciences'])]
```

between() Method

```
df[(df['Age']>=10) & (df['Age']<=50)]
```

```
df[df['Age'].between(10,50)]
```

	First Name	Gender	Start Date	Salary	Mgmt	Team
0	Douglas	Male	1993-08-06	0	True	Marketing
1	Thomas	Male	1996-03-31	61933	True	NaN
2	Maria	Female	NaT	130590	False	Finance
3	Jerry	NaN	2005-03-04	138705	True	Finance
4	Larry	Male	1998-01-24	101004	True	IT

isnull() and notnull() Method

```
df["Team"].isnull()
```

```
0    False
1     True
2    False
3    False
4    False
Name: Team, dtype: bool
```

```
df["Team"].notnull()
```

```
0     True
1    False
2     True
3     True
4     True
Name: Team, dtype: bool
```

```
~(df["Team"].isnull())
```

```
0     True
1    False
2     True
3     True
4     True
Name: Team, dtype: bool
```

tailieusharefree.blogspot.com

DataFrame Basic Functions

Head & Tail

To view a small sample of a Series or DataFrame object, use the **head()** and **tail()** methods

```
food.head()
```

	Item	Type	Price
0	Banana	Fruit	0.99
1	Cucumber	Vegetable	1.25
2	Orange	Fruit	0.25
3	Tomato	Vegetable	0.33
4	Watermelon	Fruit	3.00

```
food.head(3)
```

	Item	Type	Price
0	Banana	Fruit	0.99
1	Cucumber	Vegetable	1.25
2	Orange	Fruit	0.25

tailieusharefree.blogspot.com

DataFrame Basic Functions

Sorting

DataFrame.sort_values()

```
food.sort_values(by=['Item'])
```

	Item	Type	Price
0	Banana	Fruit	0.99
1	Cucumber	Vegetable	1.25
2	Orange	Fruit	0.25
3	Tomato	Vegetable	0.33
4	Watermelon	Fruit	3.00

```
food.sort_values(by=['Item'],ascending = False)
```

	Item	Type	Price
4	Watermelon	Fruit	3.00
3	Tomato	Vegetable	0.33
2	Orange	Fruit	0.25
1	Cucumber	Vegetable	1.25
0	Banana	Fruit	0.99

```
food.sort_values(by=['Type','Price'],ascending = False)
```

	Item	Type	Price
1	Cucumber	Vegetable	1.25
3	Tomato	Vegetable	0.33
4	Watermelon	Fruit	3.00
0	Banana	Fruit	0.99
2	Orange	Fruit	0.25

```
food.sort_values(by=['Type','Price'],ascending = [True,False])
```

	Item	Type	Price
4	Watermelon	Fruit	3.00
0	Banana	Fruit	0.99
2	Orange	Fruit	0.25
1	Cucumber	Vegetable	1.25
3	Tomato	Vegetable	0.33

tailieusharefree.blogspot.com

DataFrame Basic Functions

Select Columns and Rows

Columns

```
food[['Type','Price']]
```

	Type	Price
0	Fruit	0.99
1	Vegetable	1.25
2	Fruit	0.25

```
food['Type']
```

```
0        Fruit
1    Vegetable
2        Fruit
3    Vegetable
4        Fruit
Name: Type, dtype: object
```

Rows:

1. location => **loc**
2. integer location => **iloc**

```
>>> df = pd.DataFrame([[1, 2], [4, 5], [7, 8]],
...       index=['cobra', 'viper', 'sidewinder'],
...       columns=['max_speed', 'shield'])
>>> df
            max_speed  shield
cobra               1       2
viper               4       5
sidewinder          7       8
```

```
food.iloc[[1,2]]
```

	Item	Type	Price
1	Cucumber	Vegetable	1.25
2	Orange	Fruit	0.25

```
>>> df.loc['viper']
max_speed    4
shield       5
Name: viper, dtype: int64
```

```
>>> df.loc[['viper', 'sidewinder']]
            max_speed  shield
viper               4       5
sidewinder          7       8
```

tailieusharevn.blogspot.com

DataFrame Basic Functions

Select Columns and Rows

Slicing

- df.iloc/loc[row_start:row_end, col_start:col_end]

```
food.iloc[0:3,0:2]
```

	Item	Type
0	Banana	Fruit
1	Cucumber	Vegetable
2	Orange	Fruit

```
In [27]: s[:5]
Out[27]:
2000-01-01    0.469112
2000-01-02    1.212112
2000-01-03   -0.861849
2000-01-04    0.721555
2000-01-05   -0.424972
Freq: D, Name: A, dtype: float64

In [28]: s[::2]
Out[28]:
2000-01-01    0.469112
2000-01-03   -0.861849
2000-01-05   -0.424972
2000-01-07    0.404705
Freq: 2D, Name: A, dtype: float64

In [29]: s[::-1]
Out[29]:
2000-01-08   -0.370647
2000-01-07    0.404705
2000-01-06   -0.673690
2000-01-05   -0.424972
2000-01-04    0.721555
2000-01-03   -0.861849
2000-01-02    1.212112
2000-01-01    0.469112
```

```
food.loc[0:3,"Type":"Price"]
```

	Type	Price
0	Fruit	0.99
1	Vegetable	1.25
2	Fruit	0.25
3	Vegetable	0.33

```
food.loc[:3,:"Price"]
```

	Item	Type	Price
0	Banana	Fruit	0.99
1	Cucumber	Vegetable	1.25
2	Orange	Fruit	0.25
3	Tomato	Vegetable	0.33

tailieuscreef.blogspot.com

DataFrame Basic Functions

Describe/Count DataFrame

Generate descriptive statistics: count, mean, standard deviation, ...

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
2	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
3	R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
4	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0

```
df.describe()
```

	Number	Age	Weight	Salary
count	457.000000	457.000000	457.000000	4.460000e+02
mean	17.678337	26.938731	221.522976	4.842684e+06
std	15.966090	4.404016	26.368343	5.229238e+06
min	0.000000	19.000000	161.000000	3.088800e+04
25%	5.000000	24.000000	200.000000	1.044792e+06
50%	13.000000	26.000000	220.000000	2.839073e+06
75%	25.000000	30.000000	240.000000	6.500000e+06
max	99.000000	40.000000	307.000000	2.500000e+07

```
df['Position'].value_counts()
```

```
SG    102
PF    100
PG     92
SF     85
C      78
```

```
df['Position'].value_counts(normalize=True)
```

```
SG    0.223195
PF    0.218818
PG    0.201313
SF    0.185996
C     0.170678
```

tailieusharefree.blogspot.com

DataFrame Basic Functions

Tips & Tricks

Common attributes

```
df.shape
```

```
(458, 9)
```

```
len(df)
```

```
458
```

```
df.size
```

```
4122
```

Deal with duplicates

```
df['Position'].duplicated()
```

```
0      False
1      False
2      False
3       True
4      False
```

```
df['Position'].duplicated(keep='last')
```

```
0       True
1       True
2       True
3       True
4       True
```

```
df.drop_duplicates(subset = ["Position"])
```

	Name	Team	Number	Position	Age	Height	Weight
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0
1	Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0
2	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0
4	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0
7	Kelly Olynyk	Boston Celtics	41.0	C	25.0	7-0	238.0

Deal with null values: **df.dropna()**

```
df.dropna(subset=['Team'])
```

	Name	Team	Number	P
0	Avery Bradley	Boston Celtics	0.0	
1	Jae Crowder	Boston Celtics	99.0	
2	John Holland	Boston Celtics	30.0	
3	R.J. Hunter	Boston Celtics	28.0	

tailieusharefree.blogspot.com

DataFrame Basic Functions

Tips & Tricks

Iterate over rows

```
for index, row in df.iterrows():
  print(row['Name'])
  if index >5:
    break
```

```
Avery Bradley
Jae Crowder
John Holland
R.J. Hunter
Jonas Jerebko
Amir Johnson
Jordan Mickey
```

```
for i in range(len(df)):
  print(df.iloc[i]['Name'])
  if i>5:
    break
```

```
Avery Bradley
Jae Crowder
John Holland
R.J. Hunter
Jonas Jerebko
Amir Johnson
Jordan Mickey
```

Reference: Link

tailieusharefree.blogspot.com

DataFrame Basic Functions

Apply Function

Apply transformation to every rows or columns

DataFrame.apply(func)

	Item	YOB
0	Nga	1996
1	Bao	1997
2	Van Anh	1998
3	Truong	1999
4	Cuc	2000

```
from datetime import datetime
def calculate_age(yob):
  return datetime.now().year - yob
```

```
people['Age']= people['YOB'].apply(calculate_age)
```

```
people['Age']= people['YOB'].apply(lambda x: datetime.now().year - x )
```

	Item	YOB	Age
0	Nga	1996	26
1	Bao	1997	25
2	Van Anh	1998	24
3	Truong	1999	23
4	Cuc	2000	22

tailieushare free.blogspot.com

DataFrame Basic Functions

Apply Function

	Item	Type	Price	Quantity
0	Banana	Fruit	0.99	2
1	Cucumber	Vegetable	1.25	3
2	Orange	Fruit	0.25	4

```
def calculate_total(row):
  return row['Price']*row['Quantity']

food['Total']= food[['Price','Quantity']].apply(calculate_total,axis=1)
```

```
food.head()
```

	Item	Type	Price	Quantity	Total
0	Banana	Fruit	0.99	2	1.98
1	Cucumber	Vegetable	1.25	3	3.75
2	Orange	Fruit	0.25	4	1.00
3	Tomato	Vegetable	0.33	5	1.65
4	Watermelon	Fruit	3.00	6	18.00

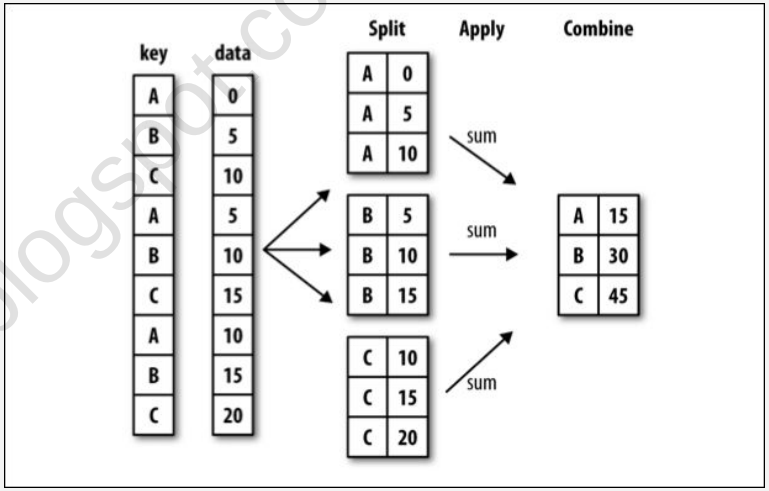
```
food['Total'] = food[["Price","Quantity"]].apply(lambda row: row[0]*row[1],axis=1)
```

tailieusharefree.blogspot.com

Groupby

split-apply-combine

- **Splitting** the data into groups based on some criteria.
- **Applying** a function to each group independently.
- **Combining** the results into a data structure.



DataFrame.groupby()

```
SELECT Column1, Column2, mean(Column3), sum(Column4)
FROM SomeTable
GROUP BY Column1, Column2
```

	Item	Type	Price
0	Banana	Fruit	0.99
1	Cucumber	Vegetable	1.25
2	Orange	Fruit	0.25
3	Tomato	Vegetable	0.33
4	Watermelon	Fruit	3.00

taileusharefree.blogspot.com

Groupby

Groupby Object

```
groups = food.groupby("Type")
groups
```

```
<pandas.core.groupby.generic.DataFrameGroupBy
```

```
len(groups)
```

```
2
```

```
groups.get_group("Fruit")
```

	Item	Type	Price
0	Banana	Fruit	0.99
2	Orange	Fruit	0.25
4	Watermelon	Fruit	3.00

```
groups.mean()
```

	Price
Type	
Fruit	1.413333
Vegetable	0.790000

```
food.groupby("Type").mean()
```

	Price
Type	
Fruit	1.413333
Vegetable	0.790000

```
groups.size()
```

```
Type
Fruit        3
Vegetable    2
dtype: int64
```

tailieusharefree.blogspot.com

Groupby

Groupby Object's Methods

Reference: Link

`GroupBy.mean`([numeric_only, engine, ...])	Compute mean of groups, excluding missing values.
`GroupBy.median`([numeric_only])	Compute median of groups, excluding missing values.
`GroupBy.min`([numeric_only, min_count])	Compute min of group values.
`GroupBy.ngroup`([ascending])	Number each group from 0 to the number of groups - 1.

`GroupBy.cumcount`([ascending])	Number each item in each group from 0 to the length of that group - 1.
`GroupBy.cummax`([axis])	Cumulative max for each group.
`GroupBy.cummin`([axis])	Cumulative min for each group.
`GroupBy.cumprod`([axis])	Cumulative product for each group.
`GroupBy.cumsum`([axis])	Cumulative sum for each group.
`GroupBy.ffill`([limit])	Forward fill the values.

Function application

`GroupBy.apply`(func, *args, **kwargs)	Apply function `func` group-wise and combine the results together.
`GroupBy.agg`(func, *args, **kwargs)	
`SeriesGroupBy.aggregate`([func, engine, ...])	Aggregate using one or more operations over the specified axis.

talieusharefree.blogspot.com

Groupby : Aggregation

Aggregation function

The **agg** method applies **multiple** aggregate operations to **different** columns

Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment
A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	13:08	Ewallet
C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29	Cash
A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	13:23	Credit carc
A	Yangon	Member	Male	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	20:33	Ewallet

```
aggregations = {
              "Invoice ID": pd.Series.nunique,
              "Total": "sum",
              "cogs": "sum"
          }
df.groupby('Branch').agg(aggregations)
```

Branch	Invoice ID	Total	cogs
A	340	106200.3705	101143.21
B	332	106197.6720	101140.64
C	328	110568.7065	105303.53

```
df.groupby('Branch').agg(OrderNum=("Invoice ID", pd.Series.nunique),
                         Total_Amount=("Total","sum"),
                         cogs=("cogs", "sum"))
```

Branch	OrderNum	Total_Amount	cogs
A	340	106200.3705	101143.21
B	332	106197.6720	101140.64
C	328	110568.7065	105303.53

tallieqsharefree.blogspot.com

Groupby : Aggregation

Groupby multiple columns

```
df.groupby(['Branch','Gender']).size()
```

```
Branch  Gender
A       Female    161
        Male      179
B       Female    162
        Male      170
C       Female    178
        Male      150
dtype: int64
```

```
df.groupby(['Branch','Gender']).sum()
```

Branch	Gender	Unit price	Quantity	Tax 5%	Total	cogs
A	Female	9029.87	909	2536.6270	53269.1670	50732.54
	Male	9595.62	950	2520.5335	52931.2035	50410.67
B	Female	8775.24	911	2520.3950	52928.2950	50407.90
	Male	9703.64	909	2536.6370	53269.3770	50732.74

tailieusharefree.blogspot.com

Merge, Join, Concatenate

Concatenate

```
pd.concat(
    objs,
    axis=0,
    join="outer",
    ignore_index=False,
    keys=None,
    levels=None,
    names=None,
    verify_integrity=False,
    copy=True,
)
```

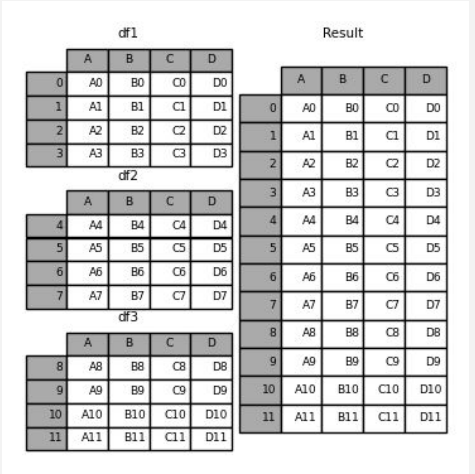
```
df1 = pd.DataFrame(
    {
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
    },
    index=[0, 1, 2, 3],
)


df2 = pd.DataFrame(
    {
        "A": ["A4", "A5", "A6", "A7"],
        "B": ["B4", "B5", "B6", "B7"],
        "C": ["C4", "C5", "C6", "C7"],
        "D": ["D4", "D5", "D6", "D7"],
    },
    index=[4, 5, 6, 7],
)


df3 = pd.DataFrame(
    {
        "A": ["A8", "A9", "A10", "A11"],
        "B": ["B8", "B9", "B10", "B11"],
        "C": ["C8", "C9", "C10", "C11"],
        "D": ["D8", "D9", "D10", "D11"],
    },
    index=[8, 9, 10, 11],
)
```

```
frames = [df1, df2, df3]

result = pd.concat(frames)
```

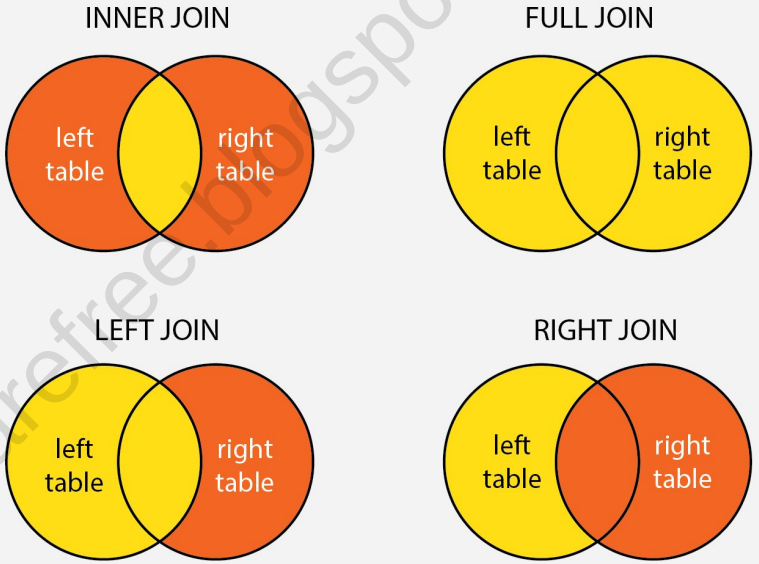


tailieusharefree.blogspot.com

Merge, Join, Concatenate

Concatenate

Common join types



Join Tables : Link

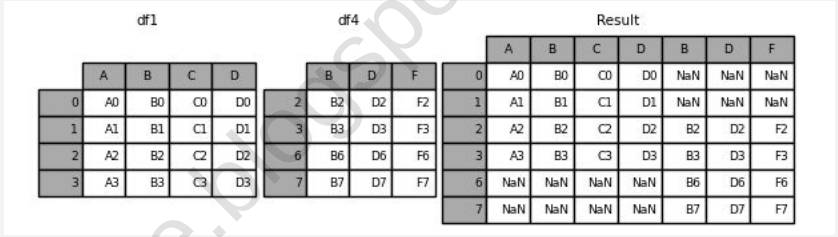
tailieusharefree.blogspot.com

Merge, Join, Concatenate

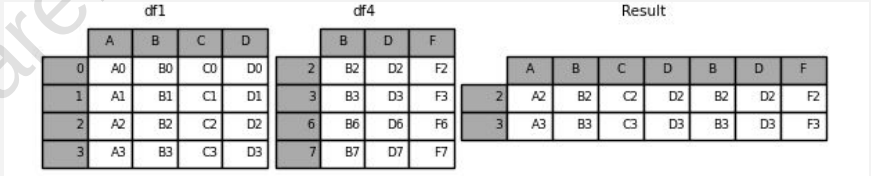
Concatenate

```
df4 = pd.DataFrame(
    {
        "B": ["B2", "B3", "B6", "B7"],
        "D": ["D2", "D3", "D6", "D7"],
        "F": ["F2", "F3", "F6", "F7"],
    },
    index=[2, 3, 6, 7],
)

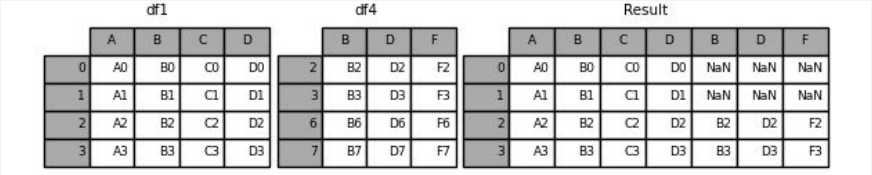

result = pd.concat([df1, df4], axis=1)
```




```
result = pd.concat([df1, df4], axis=1, join="inner")
```




```
result = pd.concat([df1, df4], axis=1).reindex(df1.index)
```



tailieusharefree.blogspot.com

Merge, Join, Concatenate

Concatenate

Ignoring indexes on the concatenation axis

```
result = pd.concat([df1, df4], ignore_index=True, sort=False)
```

df1

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3

df4

	B	D	F
2	B2	D2	F2
3	B3	D3	F3
6	B6	D6	F6
7	B7	D7	F7

Result

	A	B	C	D	F
0	A0	B0	C0	D0	NaN
1	A1	B1	C1	D1	NaN
2	A2	B2	C2	D2	NaN
3	A3	B3	C3	D3	NaN
4	NaN	B2	NaN	D2	F2
5	NaN	B3	NaN	D3	F3
6	NaN	B6	NaN	D6	F6
7	NaN	B7	NaN	D7	F7

Appending rows to a DataFrame

```
s2 = pd.Series(["X0", "X1", "X2", "X3"], index=["A", "B", "C", "D"])

result = pd.concat([df1, s2.to_frame().T], ignore_index=True)
```

df1

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3

s2

A	X0
B	X1
C	X2
D	X3

Result

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	X0	X1	X2	X3

tailieusharefree.blogspot.com

Merge, Join, Concatenate

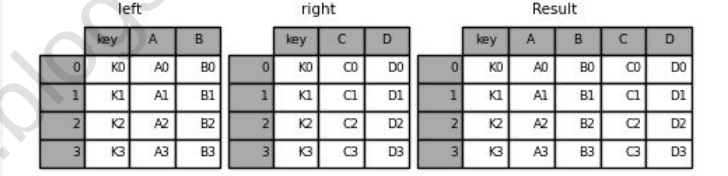
Merge

```
pd.merge(
    left,
    right,
    how="inner",
    on=None,
    left_on=None,
    right_on=None,
    left_index=False,
    right_index=False,
    sort=True,
    suffixes=("_x", "_y"),
    copy=True,
    indicator=False,
    validate=None,
)
```

```
left = pd.DataFrame(
    {
        "key": ["K0", "K1", "K2", "K3"],
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
    }
)

right = pd.DataFrame(
    {
        "key": ["K0", "K1", "K2", "K3"],
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
    }
)

result = pd.merge(left, right, on="key")
```



	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K2	C2	D2
3	K3	C3	D3

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1
2	K2	A2	B2	C2	D2
3	K3	A3	B3	C3	D3

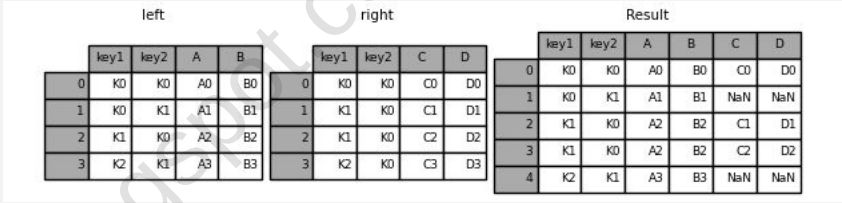
Merge method	SQL Join Name	Description
`left`	`LEFT OUTER JOIN`	Use keys from left frame only
`right`	`RIGHT OUTER JOIN`	Use keys from right frame only
`outer`	`FULL OUTER JOIN`	Use union of keys from both frames
`inner`	`INNER JOIN`	Use intersection of keys from both frames
`cross`	`CROSS JOIN`	Create the cartesian product of rows of both frames

tailieusharefree.blogspot.com

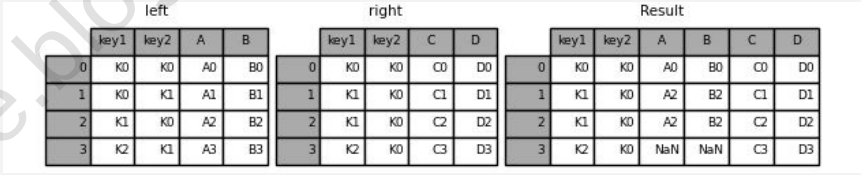
Merge, Join, Concatenate

Merge

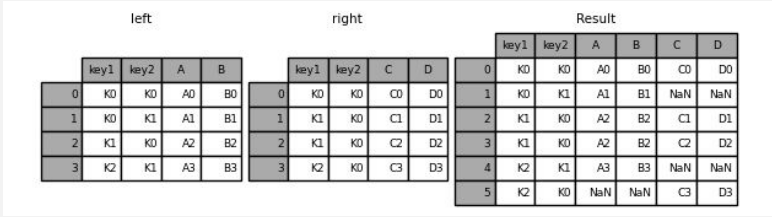
```
result = pd.merge(left, right, how="left", on=["key1", "key2"])
```



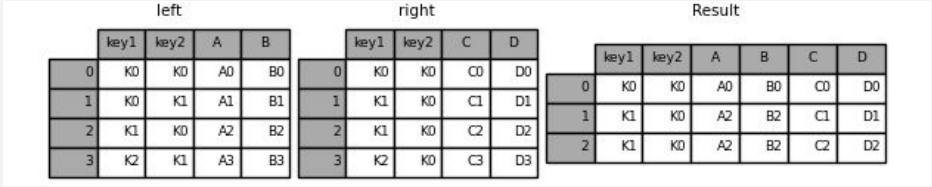

```
result = pd.merge(left, right, how="right", on=["key1", "key2"])
```




```
result = pd.merge(left, right, how="outer", on=["key1", "key2"])
```




```
result = pd.merge(left, right, how="inner", on=["key1", "key2"])
```



tailieusharefree.blogspot.com

Merge, Join, Concatenate

Merge

Checking for duplicate keys

```
left = pd.DataFrame({"A": [1, 2], "B": [1, 2]})

right = pd.DataFrame({"A": [4, 5, 6], "B": [2, 2, 2]})
```

```
In [53]: result = pd.merge(left, right, on="B", how="outer", validate="one_to_one")
...
MergeError: Merge keys are not unique in right dataset; not a one-to-one merge
```

The merge indicator

```
In [55]: df1 = pd.DataFrame({"col1": [0, 1], "col_left": ["a", "b"]})

In [56]: df2 = pd.DataFrame({"col1": [1, 2, 2], "col_right": [2, 2, 2]})

In [57]: pd.merge(df1, df2, on="col1", how="outer", indicator=True)
Out[57]:
   col1 col_left  col_right      _merge
0     0        a        NaN   left_only
1     1        b        2.0        both
2     2      NaN        2.0  right_only
3     2      NaN        2.0  right_only
```

```
In [58]: pd.merge(df1, df2, on="col1", how="outer", indicator="indicator_column")
Out[58]:
   col1 col_left  col_right indicator_column
0     0        a        NaN        left_only
1     1        b        2.0             both
2     2      NaN        2.0       right_only
3     2      NaN        2.0       right_only
```

tailieusharefree.blogspot.com

Merge, Join, Concatenate

Join on index

DataFrame.join() is a convenient method for combining the columns of two potentially differently-indexed DataFrames into a single result DataFrame

```
left = pd.DataFrame(
    {"A": ["A0", "A1", "A2"], "B": ["B0", "B1", "B2"]}, index=["K0", "K1", "K2"]
)


right = pd.DataFrame(
    {"C": ["C0", "C2", "C3"], "D": ["D0", "D2", "D3"]}, index=["K0", "K2", "K3"]
)


result = left.join(right)
```

DataFrame.join(*other,on=None,how='left', sort=False*)

left

	A	B
K0	A0	B0
K1	A1	B1
K2	A2	B2

right

	C	D
K0	C0	D0
K2	C2	D2
K3	C3	D3

Result

	A	B	C	D
K0	A0	B0	C0	D0
K1	A1	B1	NaN	NaN
K2	A2	B2	C2	D2

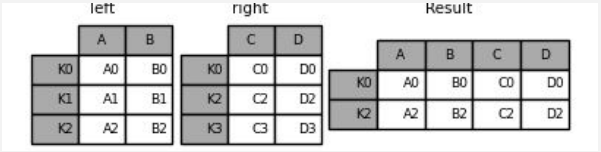
tailieusharefree.blogspot.com

Merge, Join, Concatenate

Join on index

```
result = pd.merge(left, right, left_index=True, right_index=True, how="inner")
```

```
result = left.join(right, how="inner")
```



	A	B
K0	A0	B0
K1	A1	B1
K2	A2	B2

	C	D
K0	C0	D0
K2	C2	D2
K3	C3	D3

	A	B	C	D
K0	A0	B0	C0	D0
K2	A2	B2	C2	D2

Compare Merge on Index vs Non-index Column: Link

tailieusharefree.blogspot.com